

CWSO: Phase 6 - Agentic Reinforcement Learning & Rollout Engine (NVIDIA Polar Integration)

Basierend auf den Paradigmen des NVIDIA Polar Frameworks wird der CWSO um eine asynchrone RL-Rollout-Infrastruktur erweitert. Das Ziel ist es, den CWSO in eine "Lernmaschine" zu verwandeln. Jeder Task, der in den ephemeren Shadow Workspaces ausgeführt wird, dient gleichzeitig als Datensatz für das Reinforcement Learning (z.B. GRPO) der Sub-Agenten.

1. Architektonische Kernkonzepte (Adaptiert von Polar)

- **Black-Box Harnessing:** Der CWSO muss nicht wissen, wie die internen Prompts eines spezifischen Sub-Agenten aussehen. Er schaltet sich als Proxy zwischen den Agenten und das LLM.
- **Token-Faithful Trajectory Reconstruction:** Exakte Aufzeichnung jedes Prompts, jeder Tool-Nutzung und der generierten Token inkl. Logprobs.
- **Prefix Merging / KV-Cache Sharing:** Parallele Agenten im Schwarm arbeiten oft an derselben Codebase. Der System-Prompt und der Code-Kontext (Prefix) sind identisch und müssen nur einmal im Speicher verarbeitet werden.

2. Neue CWSO-Features (Detaillierte Implementierung in Go/Rust)

Feature E: Transparent LLM Proxy & Trajectory Logger

Bisher haben Sub-Agenten direkt mit LLMs (via API) kommuniziert. Nun zwingt der CWSO alle Wasm-Micro-Agents und Docker-Container, ihren LLM-Traffic über den CWSO zu routen.

- **Implementierungsdetails (Rust/Go):**
 - **Reverse Proxy Layer:** Der CWSO startet einen internen HTTP-Server, der die OpenAI-kompatible API (/v1/chat/completions) mockt.
 - **Intercept & Log:** Jeder Request der Sub-Agenten wird abgefangen. Der CWSO speichert den exakten prompt, das system_message, die aufgerufenen tools und die vom echten Backend zurückgegebenen Token.
 - **Trajectory Builder:** Sobald ein Job abgeschlossen ist, assembliert der CWSO diese Fragmente zu einer "Token-Faithful Trajectory". Dies ist ein präziser, chronologischer Datensatz, der genau zeigt, was der Agent "gesehen" hat und welche Schlüsse er gezogen hat.
 - **Speicherung:** Ablage der Trajectories in einer hochperformanten Zeitreihen/Event-Datenbank (z.B. ClickHouse oder Apache Parquet Dateien).

Feature F: Swarm-Level Prefix Caching (KV-Cache Router)

Wenn der CWSO 50 parallele Agenten in Shadow Workspaces startet, um 50 verschiedene Bugs im selben Repository zu fixen, beginnen 90% des Kontexts (der Codebase-AST) bei allen Agenten gleich.

- **Implementierungsdetails:**
 - **Prefix Identifier Hash:** Der CWSO berechnet einen Hash für den gemeinsamen Kontext (z.B. System-Prompt + eingelesene Datei main.go).
 - **KV-Cache Offloading (z.B. mit vLLM/TensorRT-LLM):** Der Proxy sendet diesen "Prefix" *einmalig* an die GPU/LPU und hält den KV-Cache (Key-Value Cache) im VRAM "warm" (Prewarming).
 - **Differential Prompting:** Alle 50 Agenten senden ihre API-Requests. Der CWSO-Proxy nutzt den gespeicherten KV-Cache-Prefix und hängt nur noch die agenten-spezifischen Instruktionen an. Das reduziert die Inferenzkosten und Latenz des Schwarms um bis zu 80%.

Feature G: Continuous RL Rollout Loop (GRPO)

Der CWSO nutzt seine Orchestrierungs-Fähigkeiten, um "Rollout Nodes" für Reinforcement Learning bereitzustellen.

- **Implementierungsdetails:**
 - **Reward Signal Generierung:** Der CWSO liefert das perfekte, programmatische Reward-Signal für Agenten.
 - *Reward +1.0:* Der Agent löst die Aufgabe im Shadow Workspace -> Tests laufen grün -> merge_concurrent_results läuft ohne AST-Konflikte durch.
 - *Reward -1.0:* Der Agent generiert syntaktisch falschen Code oder der Merge schlägt fehl.
 - **Asynchronous Rollout Endpoints:** Implementierung der Polar-APIs direkt in den CWSO-Daemon, sodass externe RL-Trainer (wie Ray RLlib oder HuggingFace TRL) den CWSO steuern können.

3. Aktualisierte API & Tool Definitionen (Rollout Service API)

Der CWSO exponiert nun eine dedizierte REST/gRPC-API für das Training der Modelle, exakt angelehnt an die Polar-Architektur:

- **POST /rollout/task/submit**
 - *Input:* task_id, harness_config (z.B. lade Wasm-Agent Version 2.1), repository_state
 - *Verhalten:* Startet einen non-blocking Shadow Workspace. Triggert Prewarming des KV-Caches.
- **GET /rollout/task/{task_id}**
 - *Output:* Pollt den Status, liefert Partial Results und die fertig rekonstruierten Token-Trajectories zurück.
- **GET /rollout/status**
 - *Output:* Liefert Metriken über laufende Rollout-Nodes, VRAM-Auslastung und Cache-Hit-Raten.

4. Technische Bottlenecks & Mitigation (Phase 6)

1. Risiko: Proxy-Overhead & Latenz

- *Problem:* Wenn der CWSO jeden API-Aufruf abfängt, deserialisiert (JSON) und wieder serialisiert, entsteht bei hocheffizienten Hardware-Backends (wie LPUs) ein CPU-Flaschenhals.
- *Mitigation:* Implementierung des Proxys in Rust mittels hyper und Zero-Copy-Deserialization (serde_json mit Borrowing). Logs werden asynchron über Lock-Free Queues (z.B. crossbeam-channel) in einen separaten I/O-Thread geschoben.

2. Risiko: Massive Storage-Kosten für Trajectories

- *Problem:* Token-genaue Aufzeichnungen von tausenden Schwarm-Iterationen erzeugen Terabytes an JSON-Daten.
- *Mitigation:* Der CWSO speichert Trajectories nativ im Binärformat (z.B. Protobuf oder Arrow/Parquet) und nutzt LZ4-Kompression auf dem Output-Stream.

Fazit der Polar-Integration

Mit dieser Erweiterung ist der CWSO nicht mehr nur ein Werkzeug, um Code schneller zu schreiben. Er wird zu einer **Data-Engine**. Die Wasm-Agenten aus Phase 5 können tagsüber echte Entwicklungsaufgaben lösen (Inferenz) und nachts nutzt der CWSO die gesammelten Trajectories und AST-Merge-Ergebnisse, um die Agenten via GRPO (Group Relative Policy Optimization) auf genau diese spezifische Codebase zu fine-tunen.